Developing a Testing Strategy

Prepared for:

Southern Alberta Institute of Technology (SAIT) Software

CPRG-305-C Software Testing and Deployment (Fall 2024)

Asenai Shiberim

October 17,2024

Table of Contents

Contents

Section 1: Part B 3

"Identify the magnitude and sources of development risk reducible by testing......3

Section 2: **Develop a completion criterion for determining when testing is complete.** 5

Section 3: **Define how testing will be managed within your capstone project:** 5

Section 4: Describe when testing will be performed in your capstone project to give the client the best return on investment (ROI) for the project: 6

Section 5: Estimate your cost of failure. 6

Section 6: Document the following:.....7

Section 7:.) Determine which of the testing strategies presented in Chapter 4 of the resource identified at the end of the assignment are appropriate for your capstone project: 9

References 10

Section 1: Part B

"Identify the magnitude and sources of development risk reducible by testing.
Sources of Risk for a real estate website

"Functionality, security, scalibility, and user experience are key areas where rigorous testing can prevent major failures post-deployment."

Functionality risks

1. **User Role Management**: Our property management system supports multiple user roles, such as property managers, tenants, and property owners. Faulty configuration of the different portals for our various users could potentially allow unauthorized users to access restricted areas of the website, leading to security issues or incorrect functionality.
2. **Property Listings and Management**: Inaccurate descriptions of listed properties could frustrate users or cause legal issues if property availability is incorrectly reported.
3. **Lease Management**: Bugs in the lease management system (such as incorrect lease durations, payment schedules, or tenant information) could result in financial loss or legal disputes for property owners or tenants

Magnitude

These risks directly affect the major functionality of the platform and could result in serious dissatisfaction for our multiple users. It is important to undergo testing of these features to ensure that the core workflows of listing properties, managing tenants, and handling leases operate as expected.

2. Security Risks

- **Personal Data Protection**: The nature of our website deals with store sensitive tenant information (e.g., contact information, payment details, and lease agreements), vulnerabilities in the system could expose this data to unauthorized parties.

- **Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)**: Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) are two common security risks in web applications. XSS happens when attackers inject malicious code into a website, which then runs in other users' browsers, potentially stealing data or taking over accounts. CSRF tricks users into unknowingly performing actions on a website while logged in, like changing settings or transferring money. Our project is vulnerable to these risks because of the personal information contained in the different user accounts and

administrative portals. Attackers could exploit these vulnerabilities to gain unauthorized access to accounts or inject malicious code into the website.

- **User Authentication**: Failure to implement proper authentication mechanisms such as strong passwords and multi-factor authentication (could lead to unauthorized access to sensitive sections of the website, putting user data at risk.

Magnitude: Security vulnerabilities could lead to data breaches or unauthorized transactions, causing severe legal, financial, and reputational damage. The cost of resolving security issues post-deployment is typically much higher than identifying them during development. These risks associated with security vulnerabilities make security testing critical.

3. Usability Risks

- **Navigation Issues**: Developing a user-friendly interface is highly important to the success of our project. Poor navigation, broken links, or unclear user interfaces can make the website difficult for users to interact with, especially for non-technical tenants, property buyers or property managers.
- **Mobile Responsiveness**: If the website is not optimized for mobile use, this could alienate a large portion of the user base, as many tenants and property managers may access the system via smartphones or tablets.

Magnitude: Usability issues have a direct impact on user satisfaction and lead to higher abandonment rates. Poor design or navigation could cause tenants or landlords to avoid using the website altogether.

4. Performance Risks

- **Load Handling:** Our project involves traffic surge during certain periods (e.g., the first day of the month when rent payments are processed). Inadequate performance testing may result in slow load times or complete crashes during these peak periods.
- **Database Performance**: It is crucial to implement a proper database to avoid poor database performance which can slow down data retrieval, leading to long wait times for users searching for properties, tenants, or lease information.

Magnitude: Poor performance can lead to negative user experiences, slow adoption of the platform, or even system downtime during critical periods. Performance-related risks are especially significant if the system is used by multiple large property management companies with a high volume of tenants and properties.

Section 2: **Develop a completion criterion for determining when testing is complete.**

Our testing is considered complete when:

- A code coverage of 90 percent has been obtained
- There are no high severity or critical defects that could affect user experience.
- Our client and stakeholders confirm that our system and functionality meet their expectations.
- All specified requirements have been successfully tested, and every functionality has been validated.
- The system meets performance standards such as response time, load handling, and resource usage.

Section 3: **Define how testing will be managed within your capstone project:**

Testing in our Property Pro real estate website project will be managed via a collaborative method, with obligations distributed with the four group members.

- **Team roles and responsibilities** are divided as follows: one developer will manage Firebase authentication and run unit tests for login components, while another developer focuses on CRUD operations to provide data integrity in MongoDB. A tester will perform integration and black box testing to make certain continuous communication between the frontend and backend. The project manager manages the testing schedule, coordinates tasks, and make sure timely issue resolution.
- **Testing phases and scheduling** are planned into four stages: unit testing using Jest for individual components, integration testing between Firebase and MongoDB, UAT with real users (buyers, sellers, and managers) to authorise functionality, and post-deployment checking with Firebase logs and MongoDB to notice issues.
- **Risk management and prioritization** will emphasis on high-priority risks for example authentication failures and unauthorized log on, followed by medium-priority risks like property data inconsistencies affecting listings, and low-priority risks like Minor UI bugs affecting user experience but not core functionality.

- **Resources and tools** include Jest for automated tests, Firebase logs for checking login activities, MongoDB checking for database performance. All testing will be performed in a performing environment to avoid troubles to production data.
- **Test reporting and documentation** will include daily stand-ups to path progress, with final testing reports sum up failings, coverage, and remaining risks. This complete plan guarantees the platform will be strong, protected, and meet the needs of its users while reducing business risks

Section 4: Describe when testing will be performed in your capstone project to give the client the best return on investment (ROI) for the project:

In most cases, software testing should be implemented very early into the development of the software (Kara & Kara, 2021). To guarantee the best ROI for our capstone project, the testing will be implemented in bi-weekly sprints. Which began during the very earlier development stage. This approach of testing will help guide us in case of any errors or failures we find during the testing process.

For starters, implementing bi-weekly sprints of testing is crucial for our team. Since this is an agile project, the addition of new features after the requirements and design are taken care of. Will ensure that if any component of our application fails, we can catch it before. we can continue with the sprint.

As an example, one of the first features we added was authentication. This must be tested to ensure users are verified, information is stored to our database, and users have the right role-based authorization.

Investing in a solid testing strategy is crucial to deliver the best ROI to the client (*Maximizing ROI With Effective Software Testing Strategies | PractiTest*, 2024). Since our capstone team is new to testing, putting this plan into action will save us in the long run. When it comes to upfront cost, which, in this case, will be our time.

Section 5: Estimate your cost of failure.

Inability to Create or Update Listings

- **Potential Failure**: Property owners or managers are unable to create or update their property listings due to bugs in the input forms, file uploads, or backend processing.
- **Impact**: Property owners can't make their properties available, missing out on potential tenants or buyers. This causes delays and could lead to reputational damage.

- **Cost**: Direct loss of revenue for property owners and additional support costs to resolve the issue.

Scheduling or Booking Bugs for Property Viewings

- **Potential Failure**: Potential tenants or buyers are unable to schedule viewings for properties, or viewings get double-booked due to a bug in the scheduling system.
- **Impact**: Missed opportunities for both property owners and tenants/buyers, leading to frustration and loss of trust.
- **Cost**: Loss of business for property owners and costly fixes for the scheduling functionality

Broken Search or Filter Functionality

- **Potential Failure**: Users are unable to search or filter property listings effectively (e.g., by location, price range, property type), or the search results are inaccurate.
- **Impact**: Potential tenants or buyers can't find properties that match their needs, leading to frustration and a significant drop in user engagement.
- **Cost**: Fewer property views and leads, reduced client satisfaction, and a drop in user retention.

Insufficient Storage Space

- **Potential Failure**: If the backend storage system is not properly monitored, it could run out of space, leading to failed media uploads. Users may receive error messages stating that the upload could not be completed, or files may appear to upload but not be saved due to lack of space.
- **Impact**:Property owners might be unable to upload images or videos for their listings, significantly lowering the listing's appeal and the likelihood of securing a tenant or buyer.
- **Cost**: The need for urgent storage expansion, potentially requiring downtime, as well as lost opportunities due to incomplete listings and user dissatisfaction.

Section 6: Document the following:

1. How will you use positive and negative testing for risk reduction?

Positive Testing

- **Valid Inputs**: Test with correct data for creating listings, scheduling viewings, searching properties, and uploading media. Ensure expected results and proper updates across the system.
- **Expected Behavior**: Verify that actions (e.g., saving listings, booking viewings) complete successfully and reflect accurately in both frontend and backend.
- **User Feedback**: Confirm users receive clear success messages after completing actions.
- **Data Consistency**: Ensure data is stored and displayed correctly across the system

Negative Testing:

- **Invalid Data**: Input missing or invalid data (e.g., incorrect formats) and check for proper error messages.
- **Simulated Failures**: Simulate system failures (e.g., full storage or backend down) to ensure graceful error handling.
- **Edge Cases**: Test data limits (e.g., max text length, file size) and concurrent user actions to prevent conflicts.
- **Stress Testing**: Simulate heavy user load and large datasets to ensure scalability and performance under pressure.

2. What mechanism will you use to prioritize risks?

We can use a **Risk-Based Testing (RBT)** approach to prioritize:

- **Impact**: Prioritize based on how critical the issue is for users (e.g., failing lease management features would be high impact).
- **Likelihood**: Estimate the likelihood of occurrence (e.g., input validation errors may happen frequently and thus require more attention).
- **Risk Matrix**: Combine the impact and likelihood of risks to classify them as high, medium, or low priority. Address the high-priority risks first.

3. How will you use statistical analysis of defects' arrival patterns to forecast test completion?

- **Track Defect Arrival Rate**: Monitor defects found over time (daily, weekly, or per sprint) and categorize by severity (critical, major, minor). Plot on a chart to visualize trends. Early on, defect rates are high, but as testing continues and issues are fixed, the rate should decrease, signaling stabilization.

- **Monitor Defect Closure vs. Arrival**: Track defect closures alongside arrivals. As the release date nears, closures should surpass arrivals, reducing the backlog and indicating system readiness.
- **Identify Defect Aging**: Track how long defects remain unresolved. If critical defects linger, they could delay completion. Ideally, the time spent in each state should decrease, especially for high-priority issues.
- **Forecast Completion**: Use defect arrival trends to predict when the system will stabilize. A declining defect rate with most critical issues closed indicates readiness for release.
- **Respond to Rising Defects**: If defect rates don't taper off, analyze causes (e.g., new code, unresolved issues) and adjust the testing process to prevent delays.

4. How will you determine how customers will use the system to determine the appropriate testing strategy?

- **Real-World Scenarios**: Test the system in conditions that simulate actual usage, such as different device types (mobile, desktop), concurrent users (e.g., multiple property managers handling listings), and various user roles (e.g., tenants, property owners, and managers).
- **Customer Involvement**: Involve the client in user acceptance testing (UAT) to validate that the system meets their needs and that it performs in real-world scenarios.

Section 7:Determine which of the testing strategies presented in Chapter 4 of the resource identified at the end of the assignment are appropriate for your capstone project:

- **Performance testing:** By conducting performance testing, a real estate site can ensure that it meets user expectations, remains competitive, and effectively serves its audience, ultimately leading to higher engagement and conversion rates.

References

Common weakness enumeration. CWE. (n.d.). https://cwe.mitre.org/

CVE website. CVE Website. (n.d.). https://www.cve.org/

Defects. (n.d.).

https://jazz.net/devops_adoption_framework/devops.core.tech.common.extend_supp-ibm/customcategories/defects_52741311.html

Maximizing ROI with Effective Software Testing Strategies. PractiTest. (2024, September 19).

https://www.practitest.com/resource-center/blog/maximizing-roi-with-effective-testing

Software testing - test completion criterion. Tutorialspoint. (n.d.).

https://www.tutorialspoint.com/software_testing_dictionary/test_completion_criterion.htm

Team, Q. (2024, July 9). *Methods for risk-based testing*. QA and Software Testing expert.

https://qestit.com/en/blog/methods-for-risk-based-testing